第 2-5 周冲刺分工计划（6 天版）

1. 计划目标

本计划用于将原定第 2-5 周的内容压缩到 6 天内完成，目标不是“把所有功能做到完美”，而是：

- 在 6 天内完成第 2-5 周要求的核心内容。
- 保证 4 名成员都有明确、可提交到 GitHub 的开发任务。
- 保证多人并行开发时代码可以顺利合并，减少冲突。
- 在第 6 天结束前形成一个可运行、可演示、可继续迭代的基础版本。

结合当前项目情况，源码中已经具备 Electron、React、Monaco、文件树、终端、搜索、Git、Agent 等基础模块，因此本次计划重点放在：

- 统一环境
- 明确四人职责
- 快速补齐核心功能
- 做好联调与测试
- 用规范保证代码可合并

---

2. 当前进度判断

根据 `CS_-_-Monaco-AI-` 当前代码结构判断，项目已经不处于“纯方案设计阶段”，而是已经完成了较完整的基础框架，当前更接近“核心模块已搭建，进入完善和联调阶段”。

当前已具备的基础：

- 已完成 Electron + React + TypeScript + Monaco 的整体架构搭建。
- 已有编辑器、文件资源管理、终端、搜索、Git、聊天面板等前端界面。
- 已有文件系统、终端、搜索、Git、Agent 等主进程服务。
- 已具备 AI Agent 工具调用的基础能力。

当前仍需重点完成的事项：

- 统一所有成员本地环境与依赖版本。
- 验证每个模块是否“真的能跑通”，而不是只有界面和代码框架。
- 把第 2-5 周内容压缩后重新组织成可并行开发的任务。
- 制定 GitHub 协作规范，确保每个人都能提交代码且最终可合并。

---

3. 四人总分工

为了保证每个人都有独立代码任务，同时减少文件冲突，建议 4 个人按模块主责制协作。

A 同学：组长 / 架构集成负责人

负责内容：

- 统一开发环境与项目启动说明。
- 负责全局布局、状态流、模块联调。
- 负责合并分支、检查冲突、组织每日同步。
- 负责最终演示流程和阶段汇总。

主要负责文件范围建议：

- `src/App.tsx`
- `src/components/layout/`
- `src/stores/`
- `README.md`
- 项目说明类文档

GitHub 提交要求：

- 至少提交环境说明、布局联调、集成修复相关代码。
- 负责每天合并其他成员分支到 `dev`。

B 同学：编辑器与文件管理负责人

负责内容：

- 完善 Monaco 编辑器模块。
- 完善文件树、文件打开、创建、删除、重命名、刷新等功能。
- 打通“文件树 -> 打开文件 -> 编辑器展示 -> 编辑状态维护”链路。

主要负责文件范围建议：

- `src/components/editor/`
- `src/components/file-explorer/`
- `src/stores/editor.store.ts`
- `src/stores/file-tree.store.ts`
- `electron/services/file-system.service.ts`
- `electron/ipc/file-system.ipc.ts`

GitHub 提交要求：

- 至少提交编辑器功能改动 2 次以上。
- 至少完成文件管理相关功能修复或新增 2 项以上。

C 同学：终端、搜索与 Git 模块负责人

负责内容：

- 完善终端模块基本可用性。
- 完善搜索联动。
- 完善 Git 状态显示、Diff、提交、切换分支等功能。
- 配合解决与本地环境、路径、命令执行相关的问题。

主要负责文件范围建议：

- `src/components/terminal/`

- `src/components/search/`
- `src/components/git/`
- `src/stores/terminal.store.ts`
- `src/stores/search.store.ts`
- `src/stores/git.store.ts`
- `electron/services/terminal.service.ts`
- `electron/services/search.service.ts`
- `electron/services/git.service.ts`
- `electron/ipc/terminal.ipc.ts`
- `electron/ipc/search.ipc.ts`
- `electron/ipc/git.ipc.ts`

GitHub 提交要求：

- 至少提交终端或 Git 相关改动 2 次以上。
- 至少完成一个可展示的联调点，例如搜索跳转或 Git Diff 展示。

D 同学：AI Agent、测试与文档负责人

负责内容：

- 完善 Agent 对话、配置、工具调用链路。
- 测试已开发功能并记录问题。
- 编写阶段测试记录、截图说明、演示材料。
- 协助修复联调问题，但以 Agent 和测试为主。

主要负责文件范围建议：

- `src/components/chat/`
- `src/components/settings/`
- `src/hooks/useAgent.ts`
- `src/stores/chat.store.ts`
- `src/stores/settings.store.ts`
- `electron/services/agent/`
- `electron/ipc/agent.ipc.ts`
- `DOC/HW/`

GitHub 提交要求：

- 至少提交 Agent 或设置相关代码 2 次以上。
- 至少提交一次测试文档或阶段文档更新。

---

4. 6 天冲刺安排

Day 1：统一环境与任务切分

目标：

- 所有人统一 Node、npm、依赖环境。

- 跑通项目安装、启动、构建。
- 建立 GitHub 分支与协作规范。

任务分工：

- A：整理环境文档、建立 `dev` 分支、创建任务清单。
- B：检查编辑器与文件管理当前可运行状态。
- C：检查终端、搜索、Git 模块当前可运行状态。
- D：检查 Agent 配置流程和接口调用流程，整理测试清单。

当日交付：

- 所有人都能本地运行项目。
- 每个人创建自己的功能分支并完成首次提交。

Day 2：编辑器与文件管理主链路完成

目标：

- 跑通文件树、打开文件、编辑内容的主流程。

任务分工：

- A：负责联调状态管理和布局。
- B：主攻编辑器和文件树。
- C：配合文件系统接口、异常处理、搜索接口准备。
- D：编写测试用例并记录问题。

当日交付：

- “打开目录 -> 浏览文件 -> 打开文件 -> 编辑文本”可演示。

Day 3：搜索、布局、基础交互完善

目标：

- 提高编辑器与文件模块的稳定性。
- 打通搜索结果与编辑器跳转。

任务分工：

- A：统一交互逻辑和布局问题。
- B：修复编辑器交互细节和未保存状态。
- C：完成搜索联动。
- D：整理 bug 清单并跟进修复验证。

当日交付：

- 搜索联动可展示。
- 当前 bug 清单形成第一版。

Day 4：终端与 Git 模块完善

目标：

- 完成终端基本可用。
- 完成 Git 状态、Diff、提交等核心链路。

任务分工：

- A：负责各模块集成与冲突协调。
- B：修复终端接入后对编辑器和布局的影响。
- C：主攻终端、Git 功能与服务层。
- D：测试终端、Git 流程并补文档截图。

当日交付：

- 可创建终端、执行命令、查看输出。
- Git 面板可展示状态和基本操作。

Day 5：AI Agent 联调与整体收口

目标：

- 完成 Agent 基础可用链路。
- 完成主要模块联调。

任务分工：

- A：组织全项目联调与问题优先级排序。
- B：继续修复编辑器与文件模块遗留问题。
- C：继续修复终端、搜索、Git 遗留问题。
- D：主攻 Agent 对话、配置、工具调用与测试。

当日交付：

- 基本形成可演示的项目版本。
- Agent 功能具备最基础演示能力。

Day 6：统一测试、合并分支、形成演示版本

目标：

- 所有成员代码合并到 `dev`。
- 完成一轮完整测试。
- 输出演示版和阶段文档。

任务分工：

- A：合并分支、解决冲突、打出演示版本。
- B：修复最终编辑器和文件模块问题。
- C：修复最终终端、搜索、Git 问题。
- D：完成测试报告、截图、演示说明、阶段总结。

当日交付：

- `dev` 分支稳定可运行。
- 有一份可展示的阶段版本。
- 有完整的测试记录和阶段总结。

---

5. GitHub 协作与代码合并规范

这一部分必须严格执行，否则 6 天内很容易因为冲突导致进度失控。

5.1 分支规范

分支建议如下：

- `main`：稳定分支，不直接开发。
- `dev`：集成分支，所有功能先合并到这里。
- `feature/a-integration`
- `feature/b-editor-file`
- `feature/c-terminal-git-search`
- `feature/d-agent-test-doc`

规则：

- 每个人只能长期维护自己的功能分支。
- 不允许直接把代码提交到 `main`。
- 合并顺序统一为：个人功能分支 -> `dev` -> 最终稳定后再进入 `main`。

5.2 文件职责边界规范

为了减少冲突，优先按“目录主责”开发：

- A 主要修改布局、全局状态、项目说明。
- B 主要修改编辑器和文件管理相关目录。
- C 主要修改终端、搜索、Git 相关目录。
- D 主要修改 Agent、设置、测试文档相关目录。

规则：

- 修改他人主责文件前，必须先在组内说明。
- 如果必须跨模块改动，优先让主责同学一起确认。
- 不要同时多人长期修改同一个 store 或同一个组件文件。

5.3 提交规范

每次提交必须是一个明确的小任务，不要一次提交很多不相关改动。

提交信息建议格式：

- `feat: 完成文件树刷新功能`
- `fix: 修复终端切换后内容不显示问题`
- `refactor: 调整编辑器状态管理结构`
- `docs: 更新第2-5周冲刺计划`

- `test: 补充文件打开流程测试记录`

规则：

- 每人每天至少 1 次有效提交。
- 每次提交尽量只解决一个问题或一个小功能。
- 提交前先自查，避免把无关文件一起提交。

5.4 合并规范

每次合并到 `dev` 前必须满足：

- 本地项目能够正常启动或构建。
- 本次修改的主流程至少手动验证过一次。
- 已经同步 `dev` 最新代码并解决冲突。
- 提交说明清楚写明“改了什么、影响哪里、怎么验证”。

建议流程：

1. 先在自己的功能分支完成开发。
2. 执行一次本地自测。
3. 拉取最新 `dev` 并合并到自己分支。
4. 解决冲突后再次测试。
5. 提交到 GitHub。
6. 由 A 同学统一审核后合并到 `dev`。

5.5 冲突处理规范

如果出现冲突，按以下原则处理：

- 先保留功能正确的代码，再考虑风格统一。
- 冲突文件必须由相关责任人一起确认。
- 同一冲突不要反复各自覆盖，避免把别人代码冲掉。
- 冲突解决后必须重新运行项目验证。

5.6 联调规范

每天至少一次联调，联调内容包括：

- 项目是否还能启动。
- 编辑器主链路是否正常。
- 文件树操作是否正常。
- 搜索、终端、Git、Agent 是否受本次改动影响。

联调结论建议统一记录：

- 今日新增功能
- 今日修复问题
- 当前阻塞问题
- 明日任务安排

---

6. 每个人必须留下的 GitHub 贡献记录

为了保证老师或组内成员能看出每个人确实参与开发，建议明确要求：

- A 同学至少提交环境搭建、布局联调、分支集成相关代码。
- B 同学至少提交编辑器和文件管理相关代码。
- C 同学至少提交终端、搜索、Git 相关代码。
- D 同学至少提交 Agent、设置、测试文档相关代码。

建议每个人至少满足：

- 6 天内至少 4 次有效提交。
- 至少 1 次功能开发提交。
- 至少 1 次修复问题提交。
- 至少 1 次联调或文档提交。

这样在 GitHub 提交记录上，每个人都会有清晰贡献轨迹。

---

7. 每日站会制度

为了压缩时间，建议每天固定 10-15 分钟同步一次。

每个人汇报 3 件事：

- 昨天完成了什么。
- 今天准备做什么。
- 当前卡在哪里，需要谁协助。

A 同学负责每天汇总：

- 当日完成情况
- 未完成问题
- 是否影响第二天计划

---

8. 最终验收标准

6 天结束时，至少达到以下目标：

- 项目可安装依赖并成功启动或构建。
- 可打开目录并展示文件树。
- 可打开文件并在编辑器中修改内容。
- 可使用搜索功能进行定位。
- 可打开终端并执行基本命令。
- 可展示 Git 状态与基础操作。
- Agent 模块具备基础演示能力。
- GitHub 上 4 名成员均有明确代码提交记录。
- `dev` 分支代码可以稳定运行。

---

9. 建议汇报口径

如果需要对老师或组内汇报当前计划，可统一表述为：

> 结合当前项目已有基础，我们决定将原第 2-5 周内容压缩为 6 天冲刺完成。四名成员分别负责架构集成、编辑器与文件管理、终端与 Git、Agent 与测试文档四条主线，并通过统一的 GitHub 分支规范、提交规范、合并规范和联调机制来保证多人协作开发顺利进行。目标是在第 6 天形成一个可运行、可演示、可继续迭代的基础版本，同时保证每名成员都能在 GitHub 上留下清晰的代码贡献记录。